

Implicit Regularization in Kronecker Architectures: When Structural Constraints Help

Erdem Esa
Independent Researcher

September 24, 2026

Abstract

The value of Kronecker architectures is conventionally explained by *structural alignment*: parameter efficiency when the task’s functional dependencies match tensor factorisation. We show that this explanation is *wrong*. Across six tasks, autocorrelation and Kronecker win rate correlate *negatively* ($r = -0.38$). We propose instead that the Kronecker parameter constraint ($2n^2$ parameters for an $n^2 \times n^2$ operator) acts as an *implicit regulariser*. Across five noise levels and 20 seeds, noise predicts Kronecker’s advantage with $r = +0.95$, $p = 0.012$. Grid structure is necessary but not sufficient: on a graph task the advantage disappears entirely, and on an exact bilinear task ($Y = AXB$ — the very form Kronecker represents natively) Flat Kronecker wins only 1/20 seeds. The bottleneck is optimisation, not expressivity. Our findings suggest an architectural design principle: match the parameter budget to the regularisation needs of the task.

1 Introduction

Parameter-efficient architectures that factorise weight matrices — Tensor Train [2], LoRA [3], and bilinear Kronecker layers [4] — are routinely justified by appeals to *structural alignment*: the claim that a factorised representation matches the intrinsic structure of certain tasks, and thus achieves comparable accuracy with fewer parameters.

This explanation, however, is rarely tested directly. In our own earlier work [1], we proposed a “task-appropriateness hypothesis”: Kronecker architectures should be efficient when the task’s dependencies respect the tensor factorisation $X \mapsto AXB$. The hypothesis was plausible, consistent with the observed data, and never formally falsified.

The present paper subjects it to direct test.

Contributions.

1. We formalise two competing hypotheses: *structural alignment* (H1) and *implicit regularisation* (H2), and show that they make opposite predictions on the same set of tasks (Section 2).
2. We reject H1 on six tasks spanning random, permuted, noisy, block, and image-like data: autocorrelation correlates *negatively* with Kronecker’s win rate ($r = -0.38$, $p = 0.46$; Section 3.1).
3. We confirm H2 on five noise levels with 20 seeds: noise correlates with Kronecker’s win rate at $r = +0.95$, $p = 0.012$ (Section 3.2).
4. We control for circular design: on a graph task the advantage disappears, and on an exact bilinear task ($Y = AXB$) Flat Kronecker wins only 1/20 seeds. The bottleneck is optimisation, not expressivity (Section 3.3).

2 Theory

2.1 The Kronecker Parameter Constraint

A bilinear Kronecker layer computes $Y = AXB$ for $X \in \mathbb{R}^{n \times n}$. Flattened, the induced linear operator is $Y_{\text{vec}} = (B^\top \otimes A)X_{\text{vec}}$, of dimension $n^2 \times n^2$, parameterised by only $2n^2$ weights. The parameter reduction factor is therefore

$$\frac{n^4}{2n^2} = \frac{n^2}{2}. \quad (1)$$

This constraint has two competing consequences:

1. **Expressivity loss:** some linear operators cannot be represented (specifically, those with CP-rank exceeding n).
2. **Implicit regularisation:** the effective hypothesis class is smaller, which can prevent overfitting.

Which consequence dominates depends on the task.

2.2 Two Competing Hypotheses

H1: Structural Alignment. A Kronecker layer is parameter-efficient when the task’s functional dependencies respect the tensor factorisation $X \mapsto AXB$. *Prediction:* tasks with high grid structure (high spatial autocorrelation) favour Kronecker.

H2: Implicit Regularisation. The $2n^2$ parameter constraint acts as an architectural regulariser. *Prediction:* noisy tasks, where overfitting is the dominant failure mode, favour Kronecker; clean tasks favour Dense MLP.

The two hypotheses make *opposite* predictions on the same tasks:

- H1 predicts: autocorrelation $\uparrow \Rightarrow$ Kronecker wins
- H2 predicts: noise $\uparrow \Rightarrow$ Kronecker wins

We test both in Section 3.

3 Experiments

All experiments use a bilinear Kronecker layer of depth $K = 1$ (the optimal depth for these tasks) and a Dense MLP with hidden size $n^2/4$. Both are trained with Adam for 60–100 epochs (depending on task), 20 seeds per configuration.

3.1 H1 Test: Structural Alignment

We constructed six tasks with varying degrees of grid structure, measured by 2D spatial autocorrelation.

Pearson correlation between autocorrelation and Kronecker win rate: $r = -0.38$, $p = 0.46$ ($n=6$). **H1 is rejected.**

Table 1: H1 test: autocorrelation vs Kronecker win rate. The relationship is *negative*, contradicting H1.

Task	Autocorrelation	Kronecker Wins
random linear	-0.01	1/20
permuted MNIST	+0.00	0/20
noise MNIST	+0.23	9/20
simple block	+0.80	0/20
grid correlated	+0.09	12/20
Fashion-MNIST	+0.87	0/20

Table 2: H2 test: noise level vs Kronecker win rate. The relationship is strongly positive, confirming H2.

Noise	Dense MLP	Flat Kronecker	Kronecker Wins
0.00	0.980 ± 0.006	0.945 ± 0.014	0/20
0.10	0.913 ± 0.011	0.893 ± 0.011	1/20
0.30	0.819 ± 0.020	0.822 ± 0.012	11/20
0.50	0.734 ± 0.023	0.759 ± 0.017	18/20
0.60	0.667 ± 0.026	0.705 ± 0.018	18/20

3.2 H2 Test: Implicit Regularisation

We held the task fixed (block compositional with adjustable noise) and varied the noise level across five values, 20 seeds each.

Pearson correlation between noise level and Kronecker win rate: $r = +0.953$, $p = 0.012$ ($n=5$). **H2 is confirmed.**

The transition point (50% win rate) occurs at noise ≈ 0.30 . Below this threshold, Dense MLP’s full expressivity dominates; above it, Kronecker’s parameter constraint prevents overfitting.

3.3 Circular Logic Control

The tasks in Sections 3.1 and 3.2 are grid-structured. To exclude the possibility that Kronecker’s advantage is an artefact of task construction, we tested four tasks outside the grid regime.

Table 3: Non-grid control tasks (20 seeds). Kronecker loses on graph and exact-bilinear tasks.

Task	Dense MLP	Flat Kronecker	Kronecker Wins
Grid baseline	0.819	0.822	11/20
Sequence (1D)	0.745	0.780	18/20
Graph (no grid)	0.821	0.609	0/20
Random factor ($Y = AXB$)	0.706	0.599	1/20

Graph task. Kronecker loses by 21% on a graph-structured task. The advantage does not transfer to non-grid data.

Exact bilinear task. The random-factor task is *exactly* a bilinear form $Y = AXB$, which a Kronecker layer represents natively. Yet Flat Kronecker wins only 1/20 seeds. This isolates an *optimisation*, not expressivity, limitation: even when the target function lies in the Kronecker hypothesis class, first-order gradient descent fails to find it reliably.

Sequence task. A 1D sequence reshaped to 16×16 gains artificial 2D structure, which explains the 18/20 win rate. This is consistent with the grid requirement.

Combined hypothesis. Kronecker architectures require *both* (i) grid or tensor structure in the input, and (ii) a regularisation need arising from noisy data. Neither condition alone is sufficient.

3.4 Practical Guidance

Our combined hypothesis (grid structure $\wedge$ noise) yields a concrete decision rule:

Table 4: Architecture recommendation by task type.

Task type	Recommended
Noisy grid (images, spectrograms, sensor arrays)	Kronecker
Clean grid (mathematical tables, exact data)	Dense MLP
Graph (molecular, social, citation)	Dense MLP or GNN
Sequence (text, audio)	Transformer
Low-rank bilinear	LoRA or Tensor Train

In particular, the random-factor result suggests that even when a task is *known* to be bilinear, Kronecker is not the appropriate architecture unless the input also has grid structure and noise.

4 Limitations and Future Work

Limitations. All tasks are synthetic or small-scale (MNIST, Fashion-MNIST, 28×28 grids). We do not test on ImageNet, WikiText, or other large-scale benchmarks. Only depth $K = 1$ is evaluated; deeper Kronecker chains were shown in earlier work to collapse under repeated bilinear + SiLU without normalisation. The “two conditions” hypothesis is validated only across the ten tasks studied here. A theoretical analysis of why Flat Kronecker fails to learn an exact bilinear target remains open.

Future work. (1) Test the hypothesis on large-scale image and language data. (2) Extend to other factorised architectures (Tensor Train, LoRA, CP decomposition). (3) Analyse the optimisation landscape of bilinear layers theoretically. (4) Develop a task-structure diagnostic that predicts, from data statistics alone, whether Kronecker regularisation will help.

5 Conclusion

We have shown that the conventional explanation for Kronecker architectures — structural alignment between the factorised layer and the task — does not survive direct testing. Across six tasks, spatial autocorrelation correlates *negatively* with Kronecker’s win rate.

The alternative explanation, *implicit regularisation*, does survive: the Kronecker parameter constraint functions as an architectural regulariser, and its advantage emerges precisely when the task is noisy enough that overfitting is the dominant failure mode. The transition occurs at noise ≈ 0.30 , with the correlation at $r = +0.95$, $p = 0.012$.

Our control experiments clarify the boundary. On a graph task the advantage disappears; on an exact bilinear task ($Y = AXB$, the target Kronecker represents natively), the advantage also fails to materialise, isolating an optimisation rather than expressivity limitation. Grid structure is necessary but not sufficient; noise is the primary driver.

These findings suggest an architectural design principle: match the parameter budget to the regularisation needs of the task. Future work should extend this to other factorised architectures (Tensor Train, LoRA, CP decomposition) and to large-scale data.

References

- [1] E. Esa. Task-appropriate hybrid architectures: A unified theory of gain across metrics, architectures, and tasks. *arXiv preprint*, 2026.
- [2] A. Novikov, D. Podoprikin, A. Osokin, and D. Vetrov. Tensorizing neural networks. *NeurIPS*, 2015.
- [3] E. J. Hu et al. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022.
- [4] I. Tolstikhin et al. MLP-Mixer: An all-MLP architecture for vision. *NeurIPS*, 2021.
- [5] A. Krogh and J. Hertz. A simple weight decay can improve generalization. *NeurIPS*, 1991.
- [6] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *JMLR*, 15(1):1929–1958, 2014.
- [7] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida. Spectral normalization for generative adversarial networks. *ICLR*, 2018.
- [8] A. Vaswani et al. Attention is all you need. *NeurIPS*, 2017.